context-lens: measure where your Claude Code context goes

A short guide with scripts. It helps you do two things: shrink what your Claude Code workspace loads automatically, and find a fact in a decision journal without reading it whole. Version 0.1, October 2026. Free.

What this is, and what it is not

It is a method and four small tools: a budget check for the workspace, a transcript reader that shows where a session's context came from, a full-text search for a decision journal, and a drift detector for notes that disagree. They are read-only, have no dependency beyond Node 22.5 or later, and send nothing anywhere.

It is not a promise. Every number in this guide is either a **case** (one workspace, one session, one small test) or a measurement on **synthetic data**, and each comes with its source and its limits in the appendix. Nothing here says you will save a given amount. It tells you how to measure whether you did.

It is not a general introduction to AI, and it does not cover prompting beyond the eight prompts in chapter 3.

Contents

1. A measured workspace: what loads automatically, and how to shrink it.
2. Token economy and using Claude Code with intent: where a session's context comes from.
3. Free prompts: eight prompts, each with its expected effect and whether it was measured.
4. Skills: two installable skills and the YAML guard.

Appendix: sources of every figure, and known limits.

How to read it

Read chapter 1 and run the check on your own workspace first; the number you get is worth more than any number in this guide. Chapter 2 is for when your sessions feel expensive and you want to know why. Chapters 3 and 4 are reference material.

The repository holds the scripts, a ready-to-copy workspace template, the prompts as separate files and the skills as installable folders. Run `npm test` to see the whole test suite pass on your machine before you trust anything.

1. A measured workspace

The problem

Every Claude Code session starts with a context that you did not type. `CLAUDE.md` is loaded, and so is every file it pulls in with an `@path` import. Skill names and descriptions are loaded too, so the catalog can find the right skill later. If that automatic part is large, each session begins heavy, every turn re-reads it, and you pay for it before you have asked anything.

Most advice on this is a feeling ("keep CLAUDE.md short"). This chapter replaces the feeling with three numbers you can compute in a second: how many tokens load automatically, how large each skill is, and whether your skill files are even valid.

Nothing here is a promise. The figures quoted are **cases** from the author's own workspace and from synthetic test data, each with its source and its limits. Your numbers will differ. The scripts are there so that you can get yours.

The method

The method has four moves. Each one is a file or a rule in `template/`.

1. Lazy loading. `CLAUDE.md` imports only what is needed in every session (in the template: `progress.md`, about 70 tokens). Everything else is a *pointer*: one line that says when to read the file. The file is read when the condition is met, not at session start.

2. State files with an archive. `progress.md` holds what is happening now, what is blocked and the next step. When something is finished it moves to `progress-archive.md`, which nothing imports. The same pattern applies to errors: `errors.md` is an index with one line per error, and `errors-archive.md` holds the write-ups. The index is cheap to read in full; the archive is read by ID.

3. A journal you search, not read. If you keep a `DECISIONS.md`, it will outgrow any budget. Do not import it and do not read it whole. Chapter 2 and the `decisions.mjs` script cover how to find one decision by its words and open only its lines.

4. A guard on skill files. A skill is a folder with a `SKILL.md`, and the file starts with a YAML block. If the `description:` line is unquoted and contains a colon followed by a space (for example `Use when X: do Y`), the YAML is invalid and the skill silently disappears from the catalog. There is no error message; the skill just never triggers. We verified this on 2026-10-15 in the author's workspace. The fix is to wrap the description in double quotes or to avoid the colon. The check script reports this as an error so that it is caught before you wonder why a skill is not used.

The check

```
node scripts/check-workspace.mjs [--root <dir>] [--budget 8000]
```

It is read-only and has no dependency. It does the following.

- Resolves the `@` imports of `CLAUDE.md`, including imports inside imported files (up to five levels, cycle-safe), and sums an estimated token count. The estimate is **bytes divided by four**. That is a convention, not a tokenizer: real counts differ by language, by code against prose and by model.

Use the number to compare *your own files with each other and over time*, not as an absolute truth.

- Warns when the total exceeds the budget (default 8,000), and when an imported file looks like an archive or a journal.
- For every skill: checks the frontmatter exists, `name:` equals the folder name, `description:` is at least 20 characters, warns above 400 characters (the description is paid for in every session), reports the unquoted-colon error, and warns when the body exceeds 800 tokens.
- Warns when `progress.md` exceeds 1,200 tokens or `errors.md` exceeds 800.

The exit code is 1 when there is an error and 0 otherwise, so you can run it in a pre-commit hook or in CI.

On the shipped template the output is:

```
info    automatic imports: 1 file(s), ~294 tokens/session including CLAUDE.md (budget 8000)
info    skills: 1 checked
RESULT: 0 error(s), 0 warning(s)
```

That 294 is the size of an empty template. It says the *structure* is small; it says nothing about your content. The first useful run is on your own workspace.

One case: a real workspace, before and after

The author runs a long-lived, multi-project workspace. Using the same check (bytes divided by four), the automatic imports measured, on 2026-10-15:

Step	Tokens loaded per session
Before any trimming	12,596
After moving rarely-needed files from imports to pointers	8,518
After archiving finished state and shortening the rest	6,551

That is a reduction of about 48 % on one workspace, measured with a convention. Read it as a **case, not a guarantee**: it started from a workspace that had grown by accretion, and the saving you can get depends entirely on what yours imports today. The raw files are private and are not in this repository, so this figure cannot be reproduced by you. What you can reproduce is the method: run the check, trim, run it again.

What the savings did not prove

A smaller auto-loaded context is cheaper. It is not automatically *better*. Two risks are worth stating.

- **A pointer can be ignored.** If `CLAUDE.md` says "read `errors.md` when a command fails" and the model does not, you have lost information you had before. Try each pointer on a real task before relying on it.
- **Fewer tokens loaded is not fewer tokens used.** A session that reads an archive on demand can end up reading more than one that had a short summary loaded. The only honest comparison is two comparable sessions, measured with the tool in chapter 2.

Finding a fact in the journal

For a journal of decisions, the repository ships a small full-text search (`scripts/decisions.mjs`, SQLite FTS5 through `node:sqlite`, no dependency) and a drift detector.

```
node scripts/decisions.mjs search "why did we choose X"
node scripts/decisions.mjs get --file DECISIONS.md --from 120 --to 140
node scripts/decisions.mjs verify
```

`search` ranks segments (one per `### D-NNN` decision, one per section for other files) and prints `file:first-last-line`, so the answer can be checked at its source. The index is rebuilt at every call in about a tenth of a second; it is derived data and cannot be stale. `verify` runs deterministic rules and reports the rule broken with its evidence: duplicate identifiers, references to decisions that do not exist, a decision that declares it amends another which is not marked amended, dated claims older than 14 days outside the journal, and (when configured) a note that calls a project active when the latest decision put it on hold.

Measured, with their limits

The author built the search and the detector for a private workspace first. These results come from that original version:

- **Search.** On 15 questions answered with a small model (Haiku), reading through the search instead of reading the files cut tokens read by 78 % and tool calls by 59 %, with similar accuracy and one failure caused by documents that disagreed with each other. With a larger model (Sonnet) the token saving was 16 %. The raw results are private and not in this repository.
- **Detector.** On a frozen test corpus, 20 injected contradictions were detected out of 20, with no false positive, and 5 of 6 real hard contradictions were found. The tests were written by the same author as the detector, so they prove the rules are implemented, not that they generalise.

The version in this repository is a **generic rewrite** with fewer rules and its own tests (10 injections and 10 negative controls on a synthetic corpus, which it passes). The figures above must not be read as results of the rewrite. What was measured on the rewrite is in `docs/MEASUREMENTS.md`, including a lookup benchmark on a synthetic journal in which the search misses 3 of 15 questions because it matches words, not word forms (it finds "retries" but not "retry"). That limit is real: it is lexical search with no stemming and no meaning.

Do it

1. Copy `scripts/` and the parts of `template/` that you need into a project.
2. Run `node scripts/check-workspace.mjs` and write down the number.
3. Move one large import to a pointer, run it again, and note the difference.
4. Repeat while the workspace still behaves well on your real tasks.

Stop when the numbers stop moving or when a task starts failing. The goal is a workspace you understand, not a minimum.

2. Token economy and using Claude Code with intent

The problem

People report that Claude Code sessions consume more than they expected: usage limits arrive sooner than planned, the context fills up and gets compacted at inconvenient moments. The complaints are common and recent (see "Why this exists" in the README for how we looked), but a complaint is not a diagnosis. Before changing habits you want to know *where the tokens of your own sessions actually came from*.

This chapter explains the one mechanism that matters, then shows how to measure it on a real session with `scripts/measure-context.mjs`, then lists habits whose effect you can check with that same tool.

The mechanism: the context is re-read on every turn

A Claude Code session is a sequence of turns. On each turn the model is given the whole conversation so far (your messages, its replies, every tool result) and produces the next step. The conversation only grows, so **the amount read per turn grows with it**, and the total read over the session grows faster than the context itself.

A small example makes the shape clear. If the context starts at 20,000 tokens and grows by 1,000 per turn for 100 turns, it ends at 120,000, but the total read across the session is the sum of 100 numbers between 20,000 and 120,000: about 7 million. Doubling the number of turns does not double that total; it more than doubles it. What you control is how fast the context grows and where it is reset.

Two caveats. Providers cache repeated prefixes, so a large part of what is re-read is served from a cache; the transcript records it separately (`cache_read_input_tokens`). We count it in the *context size* because it is part of what the model reads; we do not claim anything about price or about how your plan's limits are computed, because we did not measure those.

The measurement

```
node scripts/measure-context.mjs                # latest session of the project in this
folder
node scripts/measure-context.mjs file.jsonl      # a specific transcript
node scripts/measure-context.mjs --window 200000 # compaction threshold used by the
simulation
node scripts/measure-context.mjs --json
```

The script reads the transcript that Claude Code stores under your home folder (`~/.claude/projects/<project>/<session>.jsonl`). It is **read-only**, makes no network call, sends nothing anywhere, and has no dependency. It reports, for each *segment* of the session (a segment ends where the context drops by more than half, which is a compaction or a clear):

- the number of turns and the context size at the start, at the end and on average;

- the **cumulative reading**: the sum of the context sizes of all turns;
- how much of the context growth was **written by the model**, split into visible text and tool calls against reasoning that is not displayed;
- how much came from **inputs**: tool results and your messages, with the five largest tools;
- a **simulation**: with the same growth but a compaction at the window you give, what would the cumulative reading have been.

How exact is it? Context sizes come from the usage fields the API returned, so they are the real numbers. The split between visible text, hidden reasoning and tool results is an **estimate** at about 3.3 characters per token, and the simulation assumes that a compaction brings the context back to the smaller of 112,000 tokens and half the window. It tells you the order of magnitude of what compaction would have changed, not an exact figure, and it ignores what is lost when information is compacted away.

One case from a long session

On one working session of the author, the script reported:

- about **59 %** of the context growth was written by the model itself (text, tool calls and reasoning), and the rest came from tool results and messages;
- a cumulative reading of about **91.7 million** tokens, against about **29 million** in the simulation with a compaction at 200,000.

This is one session. It is *not* a typical value and it is not a promise that compacting early will save two thirds for you; it shows that, in at least one real case, the model's own output was the larger source of growth, which is the opposite of the common assumption that "big files" are always to blame. Run it on your own sessions before drawing any conclusion. `docs/MEASUREMENTS.md` also lists a short session measured while this repository was built.

What the measurement points to

The tool turns a general worry into three questions:

1. **Is my starting context large?** First segment, first-turn context. If yes, chapter 1 applies: trim what loads automatically.
2. **Are tool results the biggest input?** Look at the top tools. A tool that dominates (file reads, command output, search) suggests asking for bounded reads.
3. **Is the model's own output the biggest source?** Then long reasoning and long answers are what grows your context, and the lever is how much you ask it to write, and when you reset.

Habits to try, and how to check each

Each habit is a hypothesis. The check is two comparable sessions measured with the same command.

- **Bounded reads.** Ask for a search before a read, for line ranges instead of whole files, and for filtered output. Check: the top-tools line of the next comparable session.
- **Hand-off before compaction.** When a session has done its job, write the state to `progress.md` (prompt 07) and start fresh, instead of letting the context grow to the compaction point and relying on an automatic summary. Check: the segment count and the cumulative reading.

- **Small always-loaded files.** Chapter 1. Check: `check-workspace`.
- **Short descriptions on skills.** A skill's description is loaded every session; its body only when invoked. Check: the 400-character warning.
- **Ask for what you need.** Long explanations you will not read still enter the context and are re-read on every later turn. Say when you want a short answer. Check: the "written by the model" share.

None of these is guaranteed to help in your case. A session that was dominated by something else will not change. A habit that hides information the model needed will cost you more than it saves; the clearest signal is a task that starts failing.

The objection: "memory is a waste of context"

A fair objection to everything above: if state files load into the context, aren't they exactly the waste you are trying to remove? Some people argue that agents need documentation, not memory.

The answer is that it depends on size and on when the content is loaded, and that this is measurable instead of arguable. A state file that loads in every session and that you rarely use is a cost; the same facts behind a pointer, or behind a search, are not. In the author's workspace the always-loaded part went from 12,596 to 6,551 tokens, and finding a fact through search read far fewer tokens than reading the files in a small-model test (chapter 1, with its limits). Measure your own: if `check-workspace` shows a small number and `measure-context` shows that your sessions are dominated by tool results or model output, the memory files are not your problem, and you should leave them alone.

Do it

1. Run `node scripts/measure-context.mjs` on your last long session. Write down the segment table.
2. Pick the biggest contributor. Change one thing.
3. Run it on the next comparable session and compare. One comparison is an anecdote; three are the beginning of evidence.

3. Free prompts

Eight prompts for the patterns in chapters 1 and 2. Each one says what it is for and what effect to expect. Where the effect has **not** been measured, it says so: none of these promises a saving. Measure yours with `check-workspace` (auto-loaded size) and `measure-context` (where a session's context came from).

How to use them: paste the prompt into Claude Code in your project, replacing the <PLACEHOLDERS>. Read the diff before accepting. Run the check after.

Prompt 1: Audit what loads automatically

Use when: sessions start with a large context before you type anything.

Prompt

```
List every file my CLAUDE.md imports with "@", directly or through other imports. For each one give its size in bytes and an estimate of tokens (bytes / 4). Sort by size. Do not change anything. Then tell me which three imports are the least likely to be needed in every session, and why.
```

Expected effect (not measured): a ranked list you can act on. It does not reduce anything by itself; the reduction comes from the edits you then choose. Cross-check the numbers with `npm run check`.

Prompt 2: Turn imports into read-on-demand pointers

Use when: you decided that a file is not needed in every session.

Prompt

```
In CLAUDE.md, replace the "@" import of <FILE> with a one-line instruction that says when to read it and how (for example "read only when a command fails"). Keep the file itself unchanged. Show me the diff of CLAUDE.md only.
```

Expected effect (not measured): the file stops loading at session start and is read only when the condition is met. Risk: the model may not read it when it should; test with a real task before trusting it.

Prompt 3: Keep progress.md short, archive the rest

Use when: the end of a work session, or progress.md keeps growing.

Prompt

```
Rewrite .claude/context/progress.md so it holds only: what is being worked on now, what is blocked, and the next concrete step (under 25 lines). Append everything finished to .claude/context/progress-archive.md under today's date. Do not delete any information; show me both diffs.
```

Expected effect (not measured): a smaller always-loaded file. Information is kept in the archive, which is not imported.

Prompt 4: Errors as an index plus an archive

Use when: errors.md has become a long diary.

Prompt

```
Convert errors.md into a table with one row per error: ID, symptom, fix in one line. Move the full write-up of each error to errors-archive.md under the same ID. Keep every fact; if two entries describe the same error, merge them and say so.
```

Expected effect (not measured): errors.md becomes cheap to read in full; details are fetched by ID when needed.

Prompt 5: A skill that does not show up

Use when: a skill exists on disk but never triggers or is missing from the catalog.

Prompt

```
Open the SKILL.md frontmatter of <SKILL>. Check three things: the name equals the folder name, the description is not empty, and the description is valid YAML. In particular, an unquoted description that contains a colon followed by a space is invalid YAML. If you find one, wrap the description in double quotes without changing its words, and tell me what you changed.
```

Expected effect (not measured): fixes the colon case, which we observed make a skill invisible. Other causes exist; `npm run check` reports this one deterministically.

Prompt 6: Answer from the journal without reading it

Use when: you ask "what did we decide about X" and DECISIONS.md is long.

Prompt

```
Do not open DECISIONS.md in full. Run: node scripts/decisions.mjs search "<my question>". Open only the cited lines with the "get" command. Answer with the decision id and line numbers, and say if two decisions disagree.
```

Expected effect (not measured on your files): far fewer lines read than a full read. See docs/MEASUREMENTS.md for the one synthetic case we did measure and its limits.

Prompt 7: Write the hand-off before the context is compacted

Use when: a long session approaches the point where you will compact or restart.

Prompt

```
Before we compact, write progress.md so that a fresh session can continue with no other context: the goal, what is done (with file names), what is not done, decisions taken and why, and the exact next command. Keep it under 40 lines. Do not include secrets.
```

Expected effect (not measured): after compaction the next session starts from a file you control rather than from an automatic summary alone.

Prompt 8: Ask for bounded reads and filtered output

Use when: tool results (file reads, logs, command output) dominate your context. `npm run lens` shows which tools contribute most.

Prompt

```
From now on in this session: search with grep before reading a file, read only the line ranges you need, and pipe long command output through head, tail or a filter. If you must read a large file in full, tell me its size first and ask.
```

Expected effect (not measured): smaller tool results per turn. Compare `npm run lens` on a similar session with and without this instruction; one comparison is an anecdote, not proof.

A note on pointers

Several prompts move content from "always loaded" to "read when needed". That trade works only if the model actually reads the file when it should. After applying prompt 02, give the model a real task that needs the moved file and watch whether it opens it. If it does not, make the pointer more specific (name the trigger) or put the file back.

4. Skills

Two skills ship as installable files in the repository, under `skills/`. They are not reproduced in the PDF: the files are the product, and a copy in a PDF would drift from them.

Skill	Folder	What it does
`context-budget`	`skills/context-budget/`	Walks through reducing what `CLAUDE.md` loads automatically, using the check script and reporting before and after numbers.
`decision-lookup`	`skills/decision-lookup/`	Answers "what did we decide about X" by searching the journal and opening only the cited lines, with the source.

Install

A skill is a folder containing a `SKILL.md`. Copy the folder into one of the two places Claude Code looks for skills:

- in a project: `<project>/.claude/skills/<skill-name>/SKILL.md` (shared with the team through the repository);
- for your user account: `~/.claude/skills/<skill-name>/SKILL.md` (available in every project).

For example, from a clone of this repository:

```
cp -r skills/context-budget  <project>/.claude/skills/
cp -r skills/decision-lookup <project>/.claude/skills/
```

On Windows PowerShell, `Copy-Item -Recurse skills\context-budget <project>\.claude\skills\`. Both skills call scripts from this repository (`scripts/check-workspace.mjs`, `scripts/decisions.mjs`), so copy `scripts/` into the project as well, or adjust the paths in the skill text.

How a skill is loaded, and what it costs

Only the **name and description** of a skill are loaded at the start of a session, so the catalog can match a request to a skill. The **body** is read when the skill is invoked. This has two practical consequences, both of which the check script encodes:

- a long description costs tokens in every session, so it warns above 400 characters;
- a long body costs tokens only when used, so the threshold is looser: it warns above 800 tokens.

The YAML guard

The block between the two `---` lines at the top of `SKILL.md` is YAML. In YAML, an unquoted text that contains a colon followed by a space is read as a key and a value, not as text. A description such as

```
description: Use when the user asks to rename: finds every import
```

is therefore invalid, and the skill silently disappears from the catalog; there is no warning at session start. We observed this on 2026-10-15. Two safe forms:

```
description: "Use when the user asks to rename: finds every import"
description: Use when the user asks to rename a module and update its imports
```

`node scripts/check-workspace.mjs` reports the first, unquoted form as an error. The test suite includes a test that both shipped skills pass the same check.

Write your own

Copy `template/.claude/skills/example-skill/`, rename the folder, set `name:` to the folder name, write a description of one sentence that says *when* to use it, then run the check. Keep the body to the steps; if it needs reference material, put that in a separate file and link to it so that it loads only when needed.

A caution that applies to every skill, including these two: a skill is instructions that the model follows. Read any skill you install, including ones from this repository, before you enable it.

Appendix: sources of every figure, and known limits

Every number used in this guide is listed here with where it came from. The full version, with commands, is `docs/MEASUREMENTS.md` in the repository. None of these figures is a promise about your results.

Cases from the author's own workspace (private files, not reproducible by you)

Figure	Source	Limits
Auto-loaded imports 12,596 then 8,518 then 6,551 estimated tokens per session (about 48 % lower), 2026-10-15	The workspace check (bytes / 4) before and after two rounds of trimming	One workspace; an estimate, not a tokenizer; says nothing about task quality
Search instead of reading: 78 % fewer tokens read, 59 % fewer tool calls on 15 questions (small model); 16 % fewer tokens (larger model); similar accuracy, one failure from documents that disagreed	The author's benchmark of an earlier, workspace-specific version	Private raw results; 15 questions; not a result of the generic code in the repository
Drift detector: 20 of 20 injections found, 0 false positives, 5 of 6 real hard contradictions	The earlier version, on a frozen corpus	Tests written by the detector's author; the rewrite has fewer rules
One session: about 59 % of context growth written by the model; cumulative reading 91.7 million versus about 29 million in a simulation with compaction at 200,000	An earlier version of the transcript reader	One session; the split is estimated; the simulation ignores what compaction loses

Measured on the code in the repository

Figure	Command	Limits
Template loads about 294 estimated tokens per session, 0 errors, 0 warnings	`node scripts/check-workspace.mjs --root template`	An empty template
Search on a generated journal of 58 decisions: right decision first for 12 of 15 questions, top 3 for 12 of 15; about 200 tokens returned per question against about 6,300 for reading the file whole	`node examples/bench-lookup.mjs`	Synthetic data; questions written by the author of the sample; the 3 misses are word-form mismatches (the search has no stemming)
Detector: 10 of 10 injected inconsistencies found, 0 of 10 negative controls flagged	`node --test tests/detector.test.mjs`	Synthetic and author-written: a regression test, not an accuracy measure

Known limits of the tools

- **Token counts** in the workspace check are bytes divided by four, a convention. Real tokenizers differ by language and by code against prose.
- **Transcript reader**: context sizes are the API's usage fields; the split between visible text, hidden reasoning and tool results is an estimate at about 3.3 characters per token. It reads the main conversation only, not sub-agent runs stored separately. It depends on the transcript format, which can change.

- **Search** is lexical (SQLite FTS5 with BM25): no stemming, no synonyms, no meaning. It uses `node:sqlite`, which Node still marks experimental, so pin your Node version and run the tests after upgrading.
- **Drift detector** knows explicit rules only: duplicate identifiers, references to nonexistent decisions, supersession not marked on the target, dated claims older than 14 days outside the journal, and (if configured) a note that calls a project active after a decision put it on hold. A free-form contradiction is not detected, and the detector does not say which document is right.
- **Platform**: tested on Node 22.17 under Windows 11 only.

What this guide does not claim

No saving for your workspace. No revenue. No claim that fewer tokens loaded means better results. No claim about price or about how a plan's usage limits are computed. No claim that the pointers in your `CLAUDE.md` will be followed: test them.